

Entra Hands-On Labs: Service Principals and App-to-App Authentication

Oct 10, 2026 · @Salike Hassan

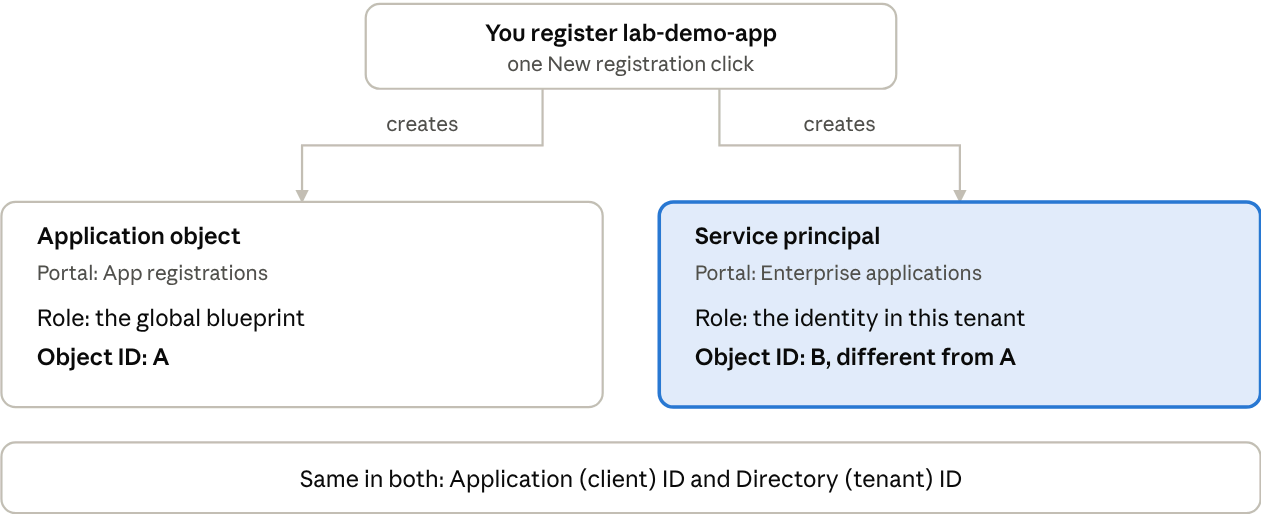
Exercise 1: one registration, two objects

When you click Register, Entra ID creates **two objects** in your tenant, not one. They are different objects, so each has its own Object ID, and the Application (client) ID is the shared thread that links them.

Steps:

1. Open entra.microsoft.com and sign in with the personal account, not your company or client one. Check the tenant name and Tenant ID on the Overview page (top right shows the account), and confirm it is your personal directory before creating anything.
2. Go to App registrations (under Entra ID, or Identity > Applications, depending on the portal layout) and choose New registration. Name it lab-demo-app, pick 'Accounts in this organizational directory only' (single tenant), leave Redirect URI empty, and select Register.
3. On the app's Overview page, copy the Application (client) ID and the Directory (tenant) ID, and also note the Object ID shown there. Keep them in a scratch note; none of them is a secret.
4. Before you click, guess: will lab-demo-app also appear under Enterprise applications? Then open Enterprise applications > All applications, clear any filters, and search for it. If found, open it and compare its Application ID and Object ID with the ones you noted in step 3.
5. On the app registration, open Certificates & secrets > New client secret, choose the shortest expiry, and copy the Value immediately because it is shown only once. Never paste it into chat or commit it to code, and delete the secret when the lab is done.
6. Tell me three things: whether the app appeared under Enterprise applications, which of the IDs matched between the two places and which differed, and anything on screen that surprised you. I will explain what you saw.

One registration creates two objects with different Object IDs



one registration, two objects

The two objects share the Application (client) ID and the tenant, but each has its own Object ID.

	Application object	Service principal
Where you see it	App registrations	Enterprise applications
What it is	The global blueprint: client ID, secrets, what the app exposes	The app's identity inside this tenant
What it is used for	Defining the app	Receiving permissions and roles
Object ID	A , the first ID you noted	B , the second ID you noted
Application (client) ID	Same	Same
Tenant ID	Same	Same

For a .NET developer, the application object is like a **class** and the service principal is like an **instance** of it. The class is defined once, in the app's home tenant. An instance is created in every tenant where the app is used.

That is why there are two objects: the same app can be used in many tenants, so there is one blueprint and one service principal per tenant. You will rely on this in the cross-tenant CI/CD topic.

Which ID goes where:

You are doing this	Use this ID
Asking Entra ID for a token	Application (client) ID
Assigning an Azure role, such as Contributor, in Bicep, ARM or Terraform	Service principal Object ID (Enterprise applications)
Editing the app definition through Microsoft Graph	Application object Object ID (App registrations)

Exercise 1: what to remember and common mistakes

- **Registration creates two objects:** the application object (blueprint) and the service principal (identity in this tenant).
- **The client ID links them,** and the Object IDs always differ.
- **Permissions go to the service principal,** because that is the identity that acts inside the tenant.

Mistake	What goes wrong	Fix
Using the app registration's Object ID for an Azure role assignment	The ID is not a service principal, so the assignment fails	Use the Object ID from Enterprise applications
Using an Object ID where a client ID is expected	The token request fails	Use the Application (client) ID
Creating an app through Microsoft Graph and expecting the Enterprise application to exist	The portal creates the service principal for you, but with Graph it is a separate step	Create the service principal explicitly

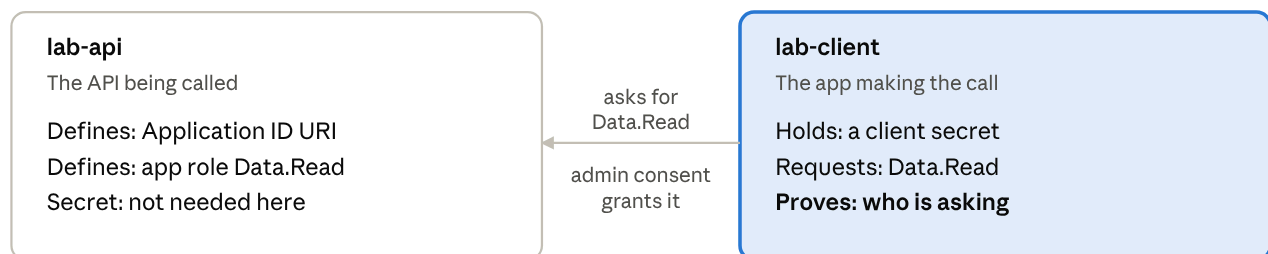
Mistake	What goes wrong	Fix
Deleting an app registration, then restoring it	Deleting also removes its service principal, and restoring the registration does not bring that back	Plan on re-creating the service principal

Exercise 2: why two app registrations

An app registration is one app's identity. In an app-to-app call, two different apps play two different parts: the one being called and the one doing the calling.

	lab-api	lab-client
Part in the call	The API being called (the resource)	The app making the call
What it defines	The Application ID URI and the app role <code>Data.Read</code>	Nothing that others use
Secret in this flow	None needed	A client secret
Question it answers	What can callers do here?	Who is asking?
Stands in for	Your secured Function App	Whatever calls it

Who owns what: the secret lives with the caller



lab-api answers: what can callers do here? lab-client answers: who is asking?

The secret answers the second question, so it belongs to lab-client.

who owns what · lab-api vs lab-client

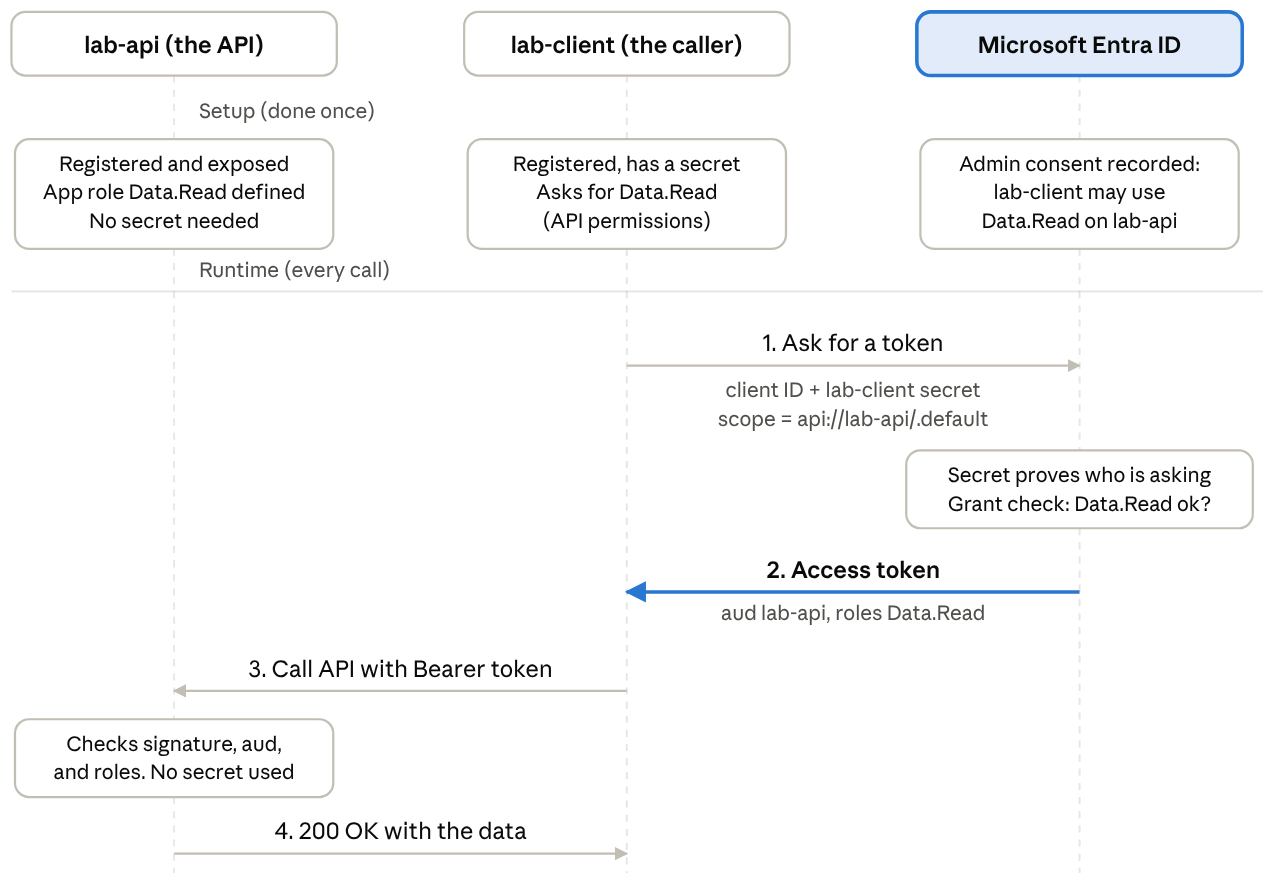
lab-client asks for the permission, an administrator grants it, and the secret stays with the caller.

Why not one app for both? Permissions are granted *between* two identities: an administrator allows one app to use a role that another app defines. With a single app there would be nothing to grant, and no way to tell the caller from the API.

Exercise 2: how setup and runtime fit together

The diagram below has two parts. The boxes at the top show the one-time setup on each app and in Entra ID. The arrows below show the four steps that happen every time lab-client calls lab-api.

Both app registrations in one flow: setup once, then four steps per call



Highlighted arrow: Entra ID issues the token.

The lab-client secret proves who is asking. lab-api never needs one, because it only checks tokens.

The role Data.Read is defined on lab-api, but the grant that allows its use is stored in Entra ID.

setup once, then 4 runtime steps

Setup happens once. Steps 1 to 4 repeat each time lab-client needs a token, and only step 1 uses a secret.

Exercise 2: the secret confusion, cleared up

The statement that caused the confusion was: *"use the client secret of lab-api, as this is the one which owns the Data.Read role."* The correct version is: **use the secret of lab-client, because it is the app asking for the token.**

The rule to remember: **the secret belongs to whoever is asking for the token.** Owning a role and asking for a token are two different jobs.

Question	Who answers it	Uses a secret in the token request?
Who is asking for the token?	lab-client, proved by its secret	Yes , lab-client's secret
What does the API offer?	lab-api defines the role Data.Read	No
Is lab-client allowed to use Data.Read ?	The admin consent recorded in Entra ID	No
Is this token genuine and meant for me?	lab-api checks the signature, aud and roles	No

Think of lab-api as a building that defines a "Data.Read" door pass. lab-client is the visitor who has been granted that pass and shows its own ID to receive it. The building's own credentials never come into it. In the PrintMagic example, the client secret belonged to PrintMagic, the caller, and not to SnapStore.

What if you used lab-api's secret? Entra ID would reject the request, because a secret only works with the client ID of the registration it was created on. Sending lab-client's ID with lab-api's secret does not match, and you would see the error AADSTS7000215 (invalid client secret). The client ID and the secret must always come from the same app registration.

So in the token request, each value comes from a specific place:

Parameter	Comes from	Why
<code>client_id</code>	<code>lab-client</code> Application (client) ID	The app asking for the token
<code>client_secret</code>	<code>lab-client</code> secret Value	Proves the caller really is <code>lab-client</code>
<code>scope</code>	<code>api:// + lab-api</code> Application (client) ID + <code>/.default</code>	The API the token is meant for
<code>grant_type</code>	<code>client_credentials</code>	The app acts as itself, with no user

The request you wrote was otherwise correct; only the owner of the secret needed fixing.

Exercise 2: step by step setup, with validation

Stay in your personal tenant. Keep a scratch note with the three IDs named below, because the token request needs them.

Step	What to do	How to validate
1. Create lab-api	App registrations > New registration. Name it <code>lab-api</code> , single tenant, no Redirect URI, then Register.	The Overview page opens. Copy its Application (client) ID and call it API_APP_ID .
2. Expose the API	On lab-api, open Expose an API, set the Application ID URI (the default <code>api://<API_APP_ID></code> is fine) and Save.	The Application ID URI shows <code>api://<API_APP_ID></code> .
3. Add the app role	On lab-api, open App roles > Create app role. Display name <code>Reader</code> , Allowed member types Applications , Value <code>Data.Read</code> , add a description, keep it enabled, then Apply.	The role is listed as enabled, with value <code>Data.Read</code> and member type Applications.

Step	What to do	How to validate
4. Create lab-client	New registration. Name it <code>lab-client</code> , single tenant, no Redirect URI, then Register.	The Overview page opens. Copy the Application (client) ID as CLIENT_ID and the Directory (tenant) ID as TENANT_ID .
5. Add a secret to lab-client	On lab-client, open Certificates & secrets > New client secret. Choose the shortest expiry and Add. Copy the Value straight away.	You copied the Value, not the Secret ID. Keep it in a secret variable, not in a file or chat.
6. Request the permission	On lab-client, open API permissions > Add a permission > My APIs > lab-api > Application permissions. Tick <code>Data.Read</code> and Add permissions.	<code>Data.Read</code> is listed with type Application, and the status says it is not yet granted.
7. Grant admin consent	Still on API permissions, select Grant admin consent for your tenant and confirm.	The status for <code>Data.Read</code> changes to Granted . Skipping this is the main reason a token arrives without roles.

Do not skip the validation column. Each check proves that one piece of the setup is in place before the next step relies on it.

Step 6 only **asks** for the permission, and step 7 is the approval that actually grants it. Nothing works until an administrator says yes.

Why an admin has to approve it

- **No user is involved.** `lab-client` calls the API as itself, so there is no signed-in person who could consent, as there is in the PrintMagic example. Microsoft's docs say application permissions are granted directly to the app by an administrator.
- **It is powerful.** An application permission works with nobody signed in and applies to the organization's data, so an admin has to approve it.
- **The grant is what makes the `roles` claim appear.** Consent records, in Entra ID, that `lab-client` may use `Data.Read` on `lab-api`. When you request a token, Entra checks that record and writes the role into the token.

What happens if you skip it

Situation	Result
Default setup (your lab)	The token request usually still succeeds, but the token has no roles claim . Microsoft's docs note that app-only tokens can be issued without one.
The API checks roles	It rejects the call, because nothing proves the caller may use Data.Read .
The API does not check roles	The call goes through anyway, which is the security gap from the "Worth checking" note.
The API requires app role assignment	The token request itself fails, because only apps with an assigned role can get a token.

So a missing **roles** claim means consent was not granted, or the role's member type is not set to Applications. Both causes are in the troubleshooting table in your doc.

Exercise 2: request the token

You need four values. Three are IDs from your scratch note and one is the secret.

Name	Value	Where to find it
TENANT_ID	Directory (tenant) ID	lab-client Overview
CLIENT_ID	lab-client Application (client) ID	lab-client Overview
API_APP_ID	lab-api Application (client) ID	lab-api Overview
CLIENT_SECRET	lab-client secret Value	Certificates & secrets, step 5

Postman. Import the curl below, then set the secret as a variable.

```
curl -X POST "https://login.microsoftonline.com/<directory-tenant-id>/oauth2/v2.0/token" \
-H "Content-Type: application/x-www-form-urlencoded" \
--data-urlencode "client_id=<lab-client application-client-id>" \
--data-urlencode "client_secret={{client_secret}}" \
--data-urlencode "scope=api://<lab-api application-client-id>/default" \
--data-urlencode "grant_type=client_credentials"
```

1. In Postman choose Import > Raw text and paste the curl above.
2. Open the request's Body tab and check it shows **x-www-form-urlencoded** with the four keys.
3. Create an environment variable named `client_secret`, paste the lab-client secret Value, and set its type to **secret**.
4. Select that environment and press Send.

What success looks like: a JSON response with `token_type` set to `Bearer`, `expires_in` (about an hour) and `access_token`. There is no `refresh_token`, because the client credentials flow never issues one.